

Java: Strukturieren mit Packages

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Typisches Java-Projekt

```
src/  
|___main/  
|___java/  
|___Book.java  
|___LibraryService.java  
|___ConsoleUI.java  
|___Member.java  
|___AppMain.java
```

Listen aus dem JDK

`java.util.List`

vs.

`java.awt.List`

Was ist ein Package in Java

Packages sind eine Ordnerstruktur unterhalb des Source-Ordners im Projekt.

- Dateisystem-Sicht: `src/main/java/de/hsbi/prog2/wuppie/`
- Java-Sicht: `de.hsbi.prog2.wuppie`

Aufgaben von Packages in Java

1. Strukturierung des Codes
2. Namensräume (Namespace)
3. Sichtbarkeit / Kapselung Package-Ebene als “Freundeskreis” von Klassen

```
de.hsbi.prog2.wuppie
```

- Umgedrehter Domain-Name: `de.hsbi` (von “hsbi.de”)
- Zusätzlich Projekt: `prog2`
- Fachliche/technische Strukturen: `wuppie`

Packages strukturieren

1. Fachliche Strukturierung: Trennung nach Domänenthemen

- `customer`: "Alles zu Kunden"
- `order`: "Alles zu Bestellungen"
- `product`: "Alles zu Produkten"

2. Technische Strukturierung: Trennung nach "Art der Aufgabe"

- `model`: Domänenobjekte
- `service`: Geschäftslogik
- `persistence`: Datenbankzugriff
- `ui`: Benutzeroberfläche
- `util`: Hilfsklassen

Praktischer Einsatz von Packages

1. Deklaration am Beginn der Java-Dateien: `package de.hsbi.prog2.library.app;`

2. Importe von Typen und Packages

- `import de.hsbi.prog2.library.model.Book;`
- `import de.hsbi.prog2.library.model.*;`
- `import static java.lang.Math.max;`

Einbahnstraße Default-Package

```
src/  
|___main/  
|___java/  
|___ConsoleUI.java  
|___de/  
|___hsbi/  
|___prog2/  
|___library/  
|___app/  
| |___LibraryService.java  
| |___AppMain.java  
|___model/  
|___Book.java  
|___Member.java
```

- Packages sind logische Container für Klassen/Interfaces und entsprechen Ordnern unterhalb des Source-Ordnern
 - Strukturierung des Codes
 - Vermeidung von Namenskollisionen (Namespaces)
 - Grenze für Sichtbarkeit (package-private)
- Zwei gängige Strukturierungsstrategien:
 - fachlich/domainorientiert (z.B. `customer`, `order`, `product`),
 - technisch (z.B. `model`, `service`, `persistence`, `ui`, `util`).
- Verwendung von Klassennamen ohne vollqualifizierten Namen per `import`
- Default-Package vermeiden



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

